Борувка на SPLA и Apache Spark. Результаты

Шальнев Владислав, Кадочников Даниил

Характеристики машины Эксперимент 1

ОС: Ubuntu 24.04.4 LTS

CPU: Intel Core i5-10300H @ 2.50 ГГц

- 4 ядра, 8 потоков (Hyper-Threading)
- L1: 256 КБ, L2: 1 МБ, L3: 8 МБ

GPU:

- NVIDIA GeForce GTX 1650 Ti Mobile (4 ГБ GDDR6);
- Intel UHD Graphics (Comet Lake-H GT2), встроенная

RAM: 16 ГБ DDR4, 3200 МГц

Программное обеспечение (Эксперимент 1)

- NVIDIA драйвер 580.126.09
- CUDA Toolkit 13.0
- SPLA (commit b81d1e2)
- Apache Spark 3.5.1
- gcc / g++ 13.3.0
- OpenJDK 11.0.30
- OpenCL 3.0
- Intel: пакет `intel-opencl-icd` 23.43.27642.40-1ubuntu3 (Neo)
- CMake 3.28.3

Данные Эксперимент 1

Граф	Количество вершин	Количество рёбер
amazon0601	403 394	2 443 408
citationCiteseer	268 495	1 156 647
coPapersDBLP	540 486	15 245 729
in-2004	1 382 908	13 591 473
cit-Patents	3 774 768	16 518 947

Источники: A High-Performance MST Implementation for GPUs 2023, SuiteSparse Matrix Collection

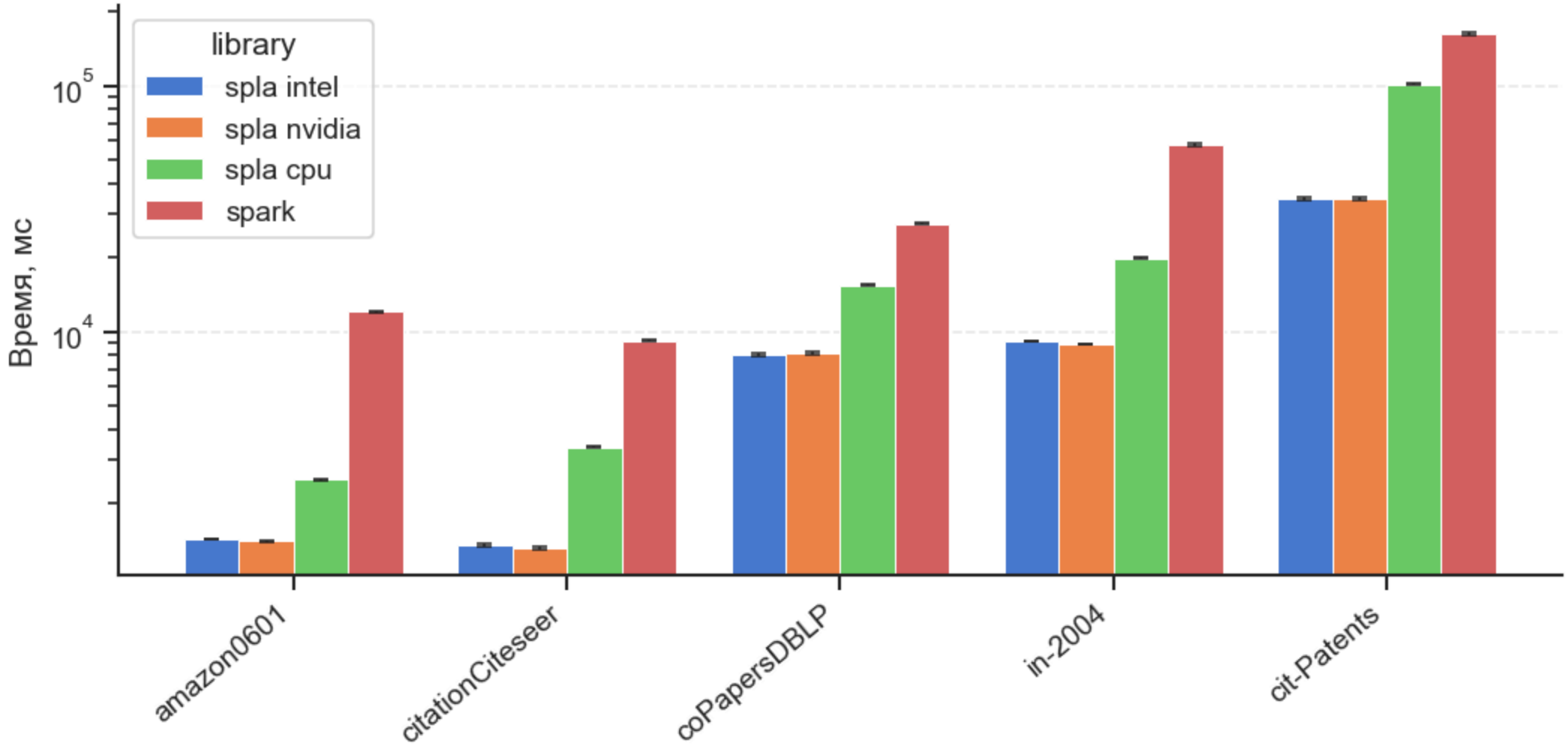
Эксперимент 1

Цель: сравнить время работы алгоритма Борувки на SPLA (CPU, NVIDIA OpenCL и Intel OpenCL) и Apache Spark на реальных графах разного размера

Ход экспериментов

- 15 запусков алгоритма на каждом графе и библиотеке, 5 прогревов
- Реализация на Spark запускается с `--cores=8`
- Вычисление среднего времени и доверительных интервалов
- Визуализация и анализ результатов

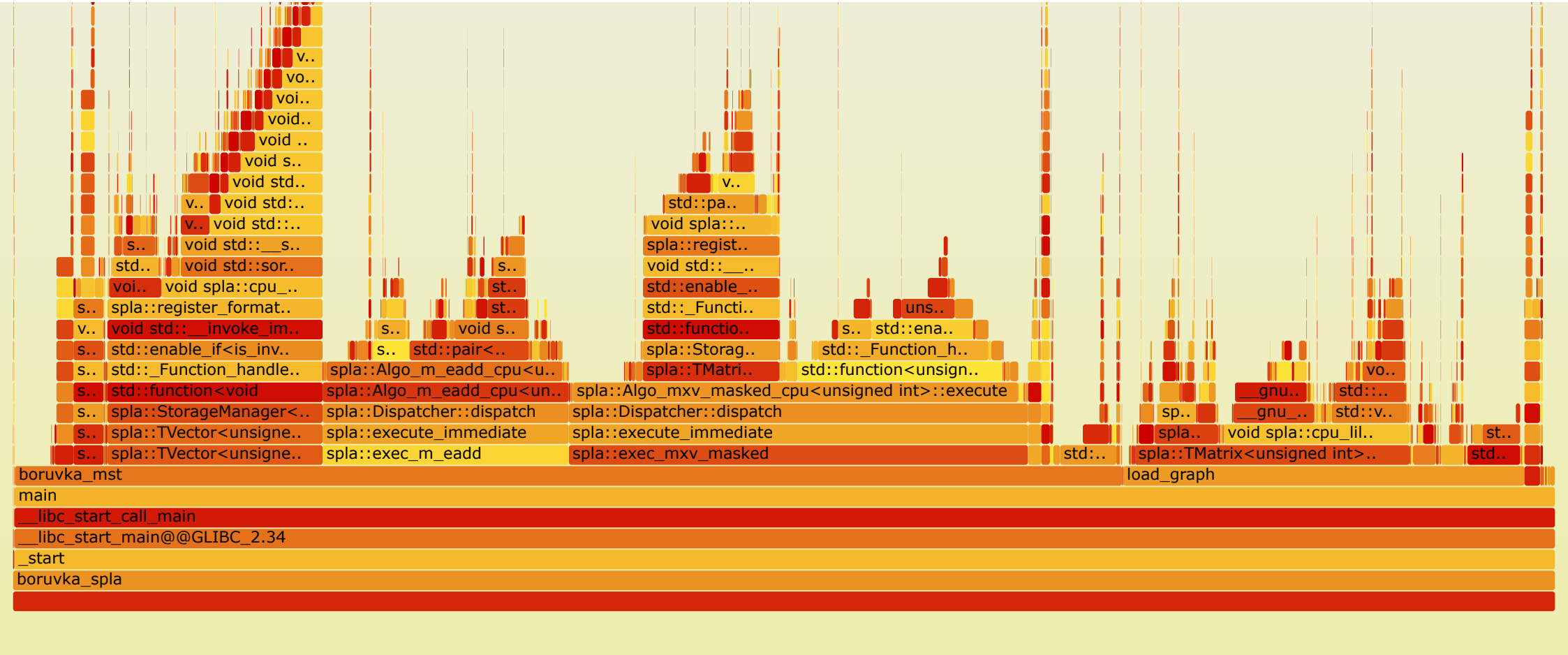
Эксперимент 1



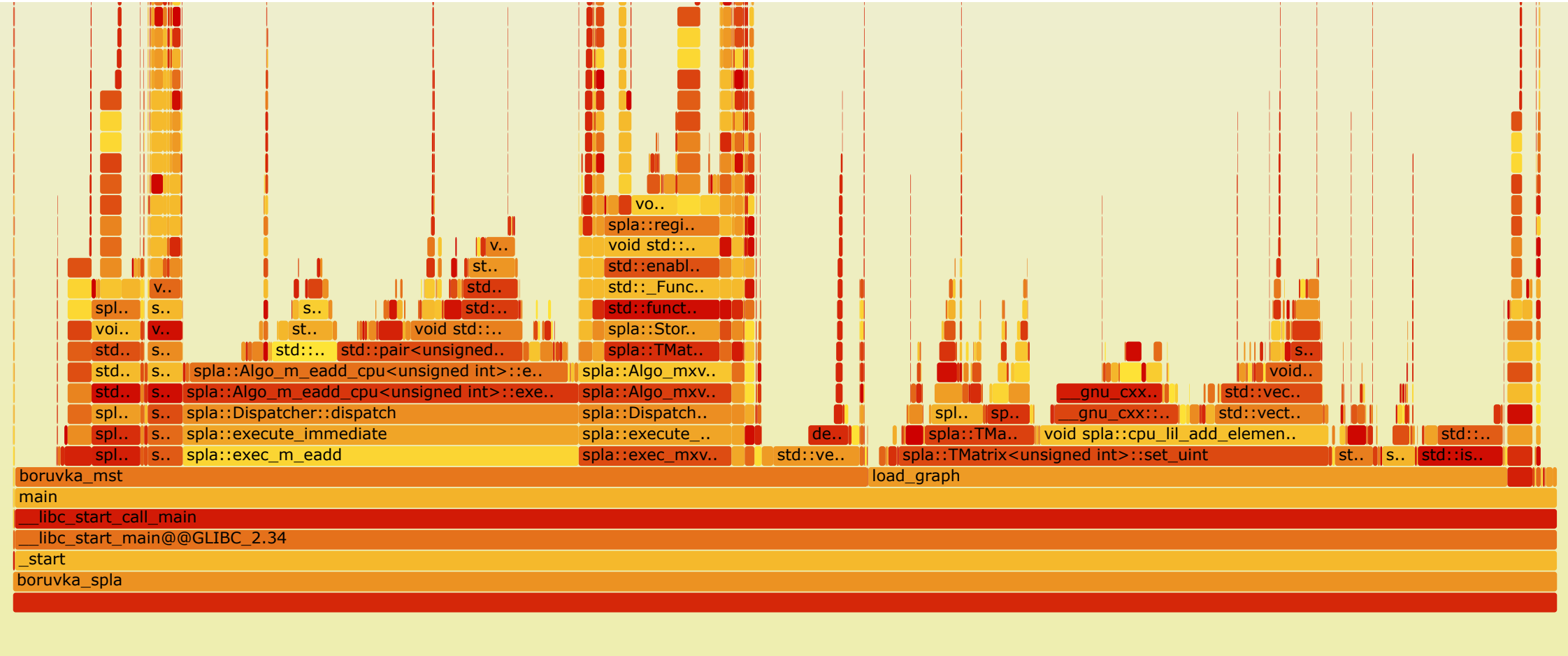
Результаты

- **Intel OpenCL и NVIDIA OpenCL:** время на всех пяти графах **близкое** (расхождение порядка **1–3%**).
- **Относительно SPLA CPU** оба OpenCL-бэкенда **быстрее** примерно **в 1,6–2,8 раза** (минимум на coPapersDBLP и cit-Patents, максимум на citationCiteseer и in-2004).
- **Относительно Spark** лучший SPLA (OpenCL) **быстрее** примерно **в 3,5–9 раз**; наибольший запас на **amazon0601** и **citationCiteseer**, наименьший на **coPapersDBLP** (~**3,5×**).

Профилирование SPLA (in-2004.mtx, CPU only)



Профилирование SPLA (in-2004.mtx, Intel OpenCL)



Анализ результатов

- **SpMV (`exec_mxv_masked`)** — по профилю один из самых затратных шагов; **на OpenCL даёт значительное ускорение** относительно SPLA CPU
- `exec_m_eadd` (в т.ч. построение рабочей S из A) — **существенная доля** времени; **OpenCL-реализации нет**, выполнение на CPU
- **Чего не хватало в SPLA**: **пользовательских типов** и полукольца на кортежах (w, idx) без упаковки в `uint32`; примитивов **scatter** (аналог `FastAssign`) и **gather** (`t[i]=f[f[i]]`) `select` для фильтрации матрицы по (i, j, val); **маскированного eadd**

Характеристики машины Эксперимент 2

ОС: Windows 10, 64-разрядная

CPU: 12th Gen Intel Core i5-1235U @ 1.30 GHz

- 10 ядер (2P + 8E), 12 потоков

GPU: Intel Iris Xe Graphics (128 MB, интегрированная)

RAM: 32 ГБ

Данные Эксперимент 2

Тип	Граф	Вершины	Рёбра
Road	road-luxembourg-osm	114 599	119 666
Road	road-usroads-48	126 146	161 950
Road	road-roadNet-PA	1 087 562	1 541 514
Geometric	delaunay_n17	131 072	393 176
Geometric	delaunay_n18	262 144	786 396
Geometric	delaunay_n20	1 048 576	3 145 686
Social	soc-LiveMocha	104 103	2 193 083
Social	ca-MathSciNet	332 689	820 644
Social	soc-lastfm	1 191 805	4 519 330

Источник: Network Repository

Эксперимент 2

Цель: исследовать влияние типа графа на относительную производительность реализаций алгоритма Борувки на SPLA (CPU) и Apache Spark

Вопрос: зависит ли соотношение времени работы SPLA и Spark от структурных свойств графа?

Типы графов:

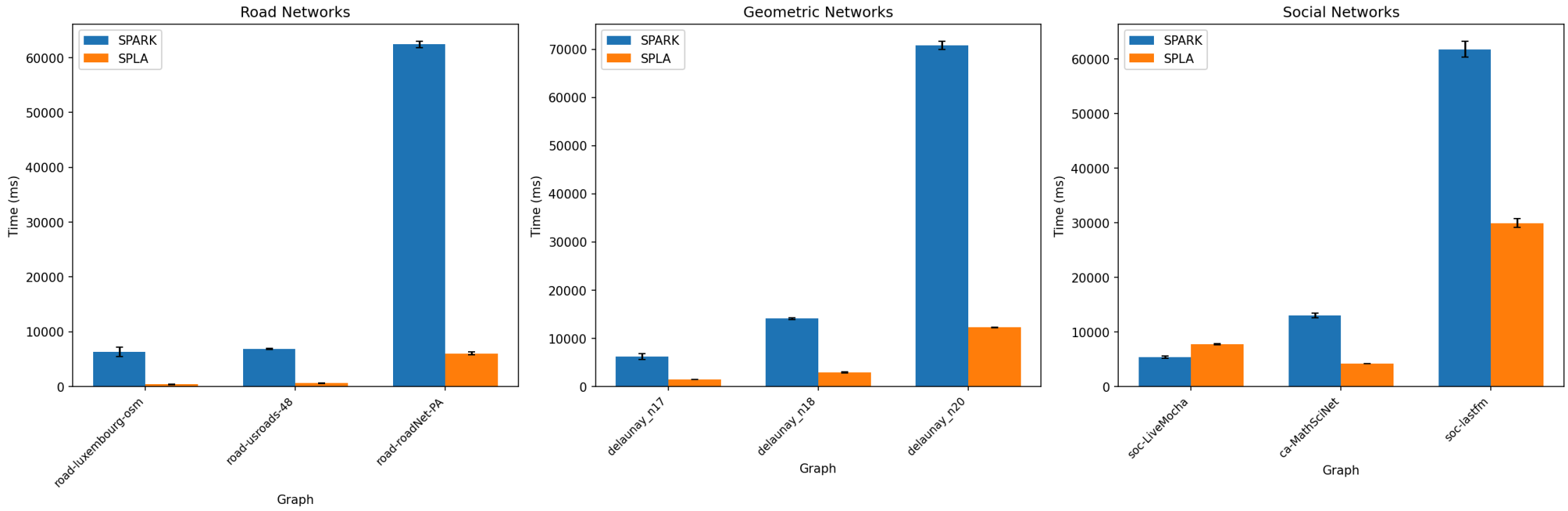
- **Road** — разреженные дорожные сети, много итераций Борувки
- **Geometric** — триангуляции Делоне, средняя плотность
- **Social** — социальные/коллаборационные сети, высокая плотность, мало итераций

Ход эксперимента 2

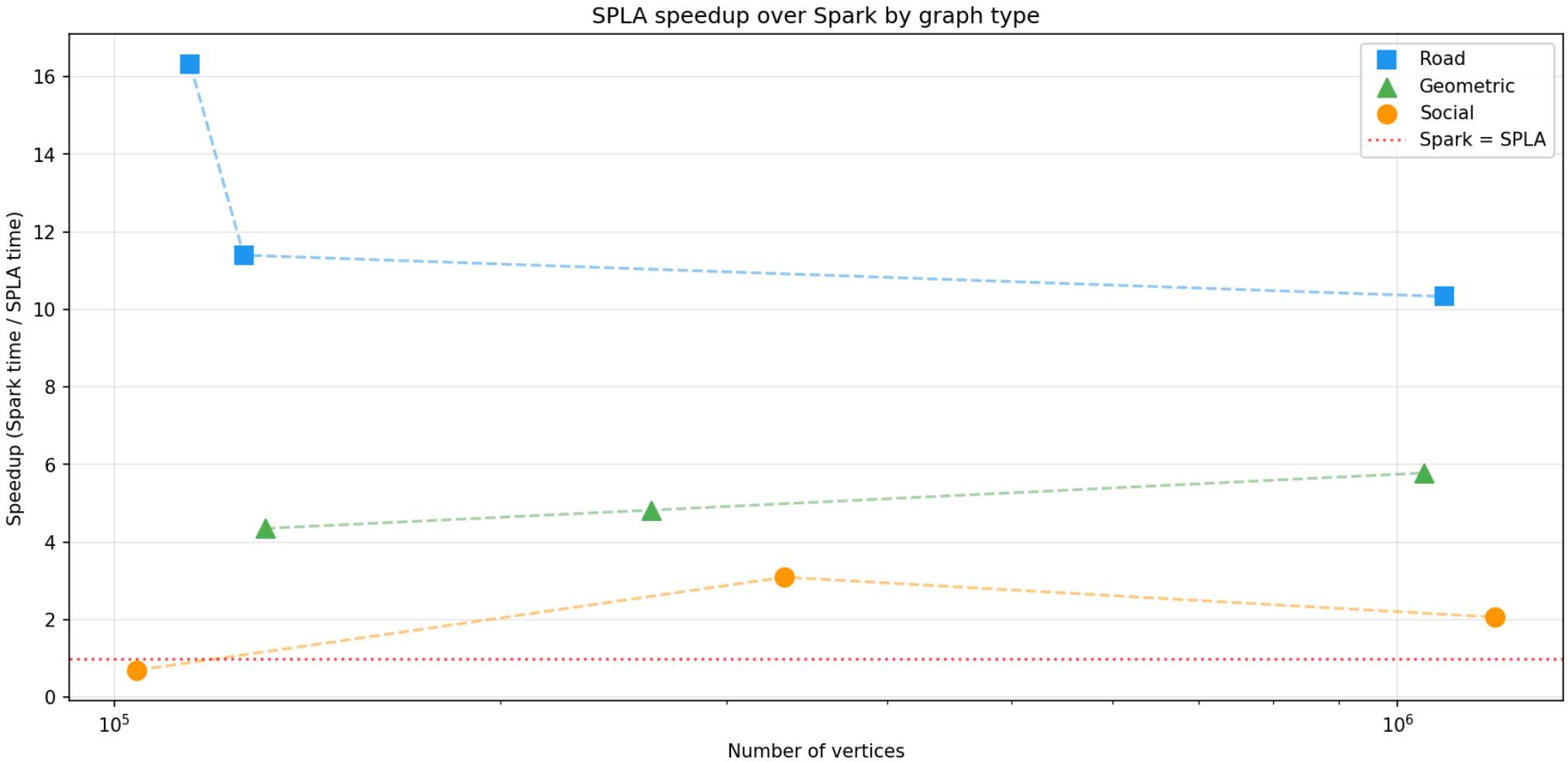
Влияние типа графа

- 5 запусков алгоритма на каждом графе и библиотеке, 3 прогрева
- Сравнение SPLA (CPU) и Spark
- Визуализация и анализ результатов

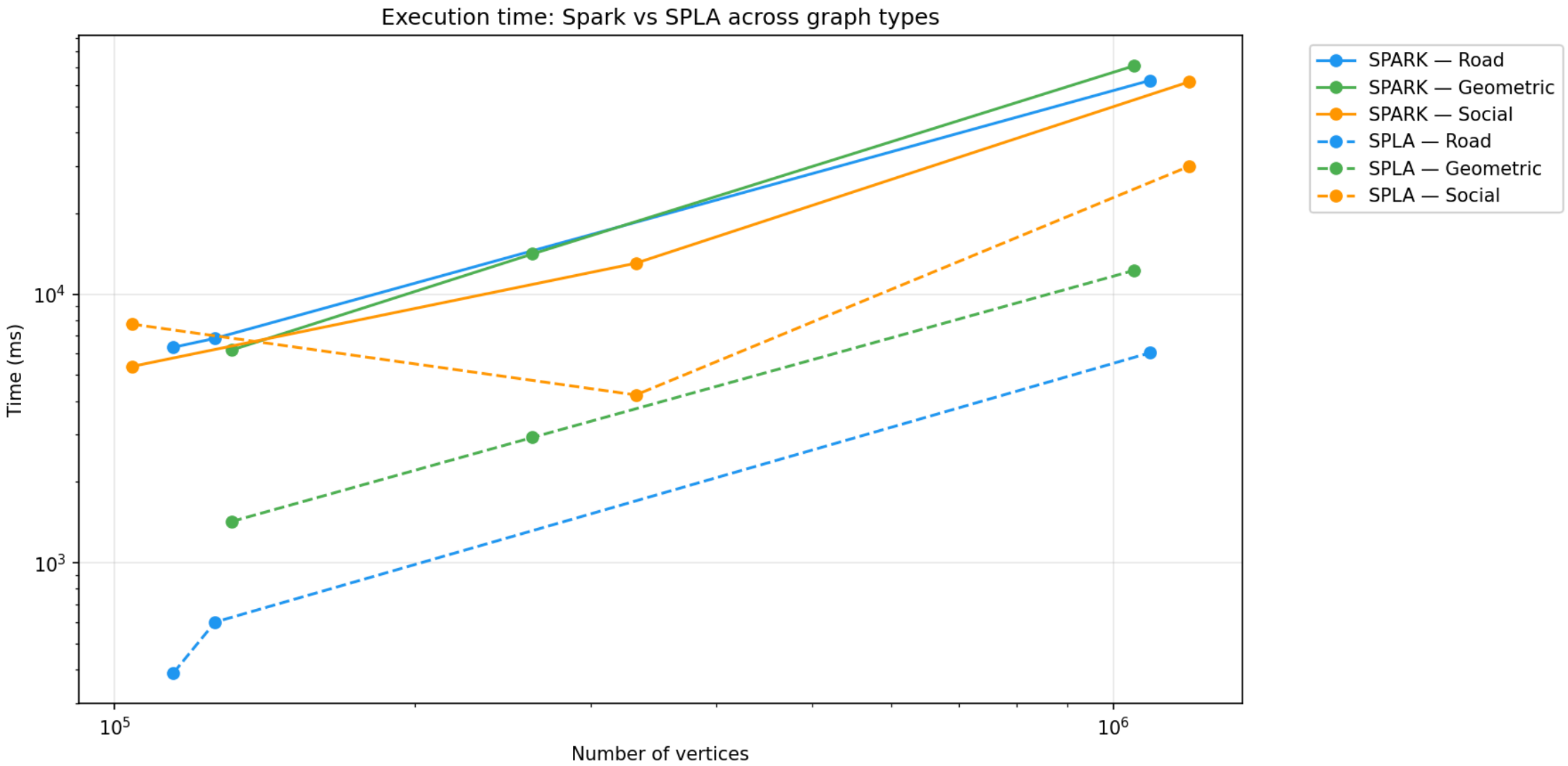
Эксперимент 2 — Сравнение по типу графа



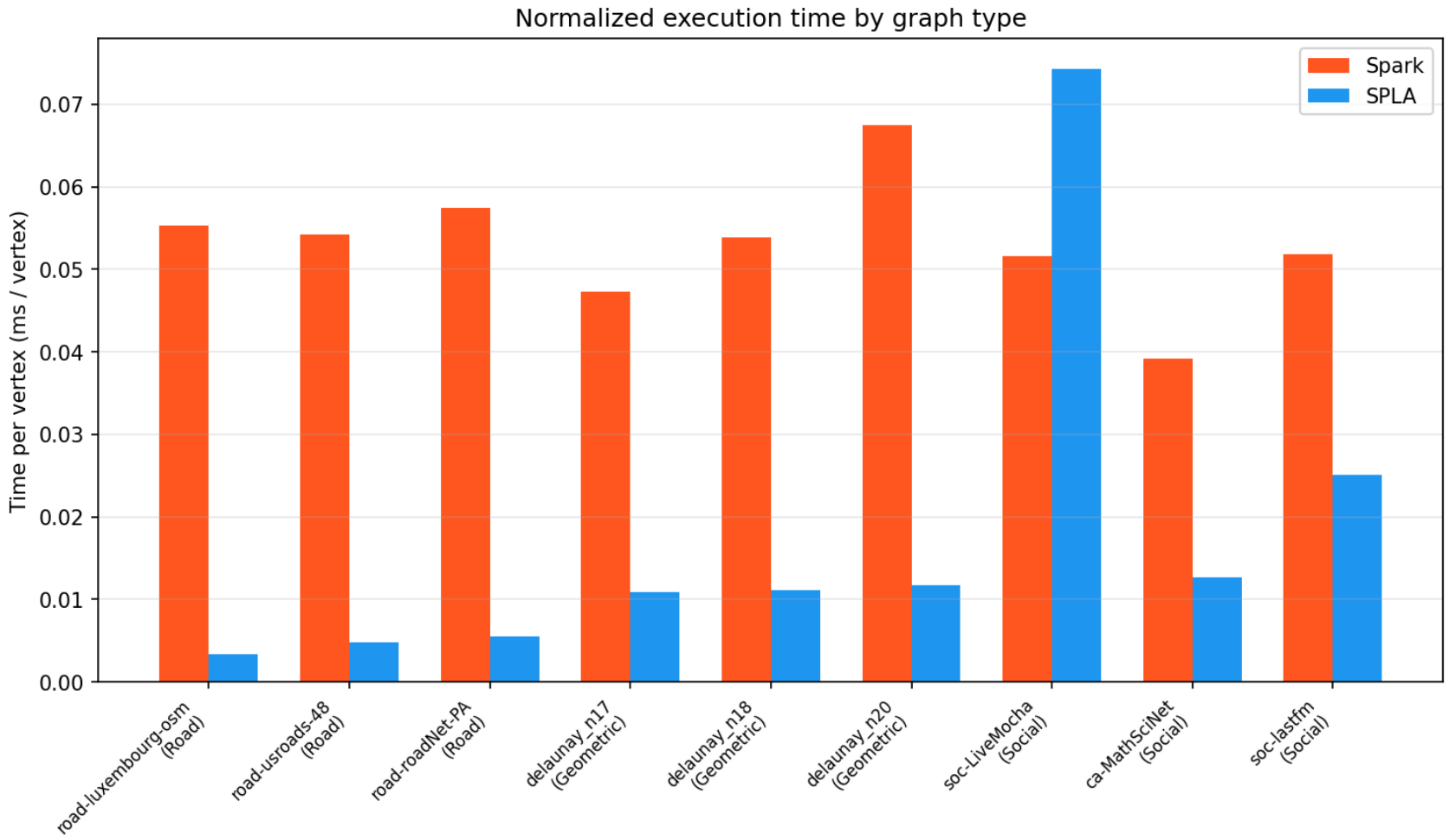
Эксперимент 2 — Ускорение SPLA относительно Spark



Эксперимент 2 — Время по размеру и типу графа



Эксперимент 2 — Нормализованное время (ms/вершину)



Выводы Эксперимент 2

1. **Тип графа влияет на соотношение производительности** SPLA и Spark
2. **Road-графы** — наибольший разрыв: Spark тратит больше на многочисленные итерации
3. **Social-графы** — наименьший разрыв: мало итераций, overhead Spark менее заметен
4. **Главный фактор** — число итераций алгоритма Борувки, которое определяется структурой графа
5. При выборе реализации MST следует учитывать **тип входных данных**